ADS2002 Final Project Report

Automated Detection of Malaria Infected Cells

Lecturer:
Dr. Ashish Dutt

Members:
Chow Chun Kei (33520771)
Esher Loh Ci Ying (34392793)
Gue Zhen Xue (33521352)
Lim Hui Shawn (34017259)
Wesley Kong Zhe Hung (33521743)

Date of Submission:
27th October 2025

Source Code:
Link to GitHub repository

Table of Contents

Table of Contents 2
Executive Summary 4
Problem Statement 4
Methodology 4
Findings 5
1. Introduction 6
Problem domain 6
Data overview 6
Expectation and approach 6
2. Data Quality 8
Data preview 8
Image properties 8
Quality issue: Mislabelled images 8
Data preprocessing and augmentation 9
3. Model Development 10
Overall Approach in Exploratory Data Analysis (EDA) 10
Class Distribution Analysis 10
Pixel Intensity Distribution Analysis & KL-Divergence Analysis 11
Principal Component Analysis (PCA) 11
Modelling Approaches 12
Baseline Model 12
Advanced Training Backbone 13
Transfer learning 15
Key Assumptions and Potential Limitations of Models 16
4. Results 18
Training Dynamics 18
Warm-up Phase (Epochs 1-5) 18
Fine-tuning Phase (Epoch 6 onwards) 19
Convergence and Stability 19
Model Performance (Validation) 19
Model Performance (Testing) 21
5. Conclusion 23

References 24
Appendix A: Advanced Training Backbone 26
Appendix B: Transfer learning 28
Appendix C: Group Assessment Form 29

Executive Summary

Problem Statement

The goal of this project is to identify malaria-infected cells from microscopic images. The dataset used for this project originates from the National Institutes of Health (NIH) which is a part of the U.S. Department of Health and Human Services. Current manual classification methods such as microscopic examination of Giemsa-stained blood smears are labour-intensive, time-consuming and prone to human error. These resources are often inaccessible in remote areas, thus resulting in preventable deaths. Therefore, this project aims to develop a tool can assist healthcare professionals in rapid and accurate malaria diagnosis.

Methodology

This project aims to develop an image classification model that can accurately identify malaria-infected cells from microscopic images. The dataset from the NIH contains 27558 images of infected and uninfected cells which are divided equally into 2 folders which represents each class.

Image pre-processing and normalization was applied to the images to ensure that all images are consistent, comparable, and numerically stable for efficient training. First, the images were then resized to a uniform resolution of 64×64 pixels so that the model could effectively process them in batches. Then, the pixel intensities were standardized to a range of [-1, 1] to improve training stability and model convergence.

Exploratory data analysis (EDA) was conducted to determine whether the distribution of the two classes was balanced as an imbalanced dataset might result in a biased model. Then, we investigated the difference between the pixel intensity distributions of the parasitized and uninfected cell image using the Kullback-Leibler (KL) divergence. Finally, we applied Principal Component Analysis (PCA) to visualize the overall data distribution in a lower-dimensional space. By projecting the high-dimensional image features onto the first two principal components (PC1 and PC2), we were able to observe how the data points from different classes are distributed.

For model development, several deep learning architectures were trained and evaluated for malaria cell image classification. The baseline model was a Convolutional Neural Network (CNN), as it can capture spatial and structural patterns in cell images. To enhance performance and generalization, advanced training techniques such as early stopping, gradient clipping, label smoothing, cosine annealing learning rate scheduling, and mix-up data augmentation were integrated. Furthermore, transfer learning was applied using pretrained models including ResNet18, ResNet50, DenseNet121, and ConvNeXt-base, allowing the models to leverage prior knowledge from large-scale datasets. Model performance was assessed using accuracy, precision, recall, F1-score, and AUC metrics, ensuring a comprehensive evaluation of predictive performance and robustness across architectures.

Findings

From the EDA, we managed to obtain insight into the characteristics of the malaria cell image dataset. First, the EDA revealed that the dataset was well-balanced, with the chi-squared test ($p = 1.0$) confirming no significant difference in class frequencies between parasitized and uninfected cells, ensuring unbiased learning during model training. The pixel intensity distribution analysis indicated a statistically significant difference between the two classes, suggesting that infected and uninfected cells exhibit distinguishable pixel-level features. The KL-divergence value of 0.0594 showed minimal divergence, implying similar but subtly distinct pixel intensity distributions between classes. Furthermore, PCA demonstrated a high degree of overlap between classes in two-dimensional space, indicating that the data were not linearly separable. This observation justified the adoption of non-linear and deep learning models capable of learning complex, high-dimensional feature representations.

In the modelling phase, the baseline CNN achieved strong performance with a validation accuracy of 98.85% and a ROC-AUC of 0.9970, confirming its suitability as a foundational model. Model generalization and robustness were further enhanced with the integration of advanced training techniques such as early stopping, gradient clipping, label smoothing, cosine annealing scheduling, and mix-up augmentation. Through transfer learning, all pretrained architectures achieved high test accuracies between 98.28%–98.80% and ROC-AUC values above 0.996. Among them, DenseNet121 demonstrated the best accuracy at 98.80%, while ResNet18 achieved the highest recall (99.38%), making it particularly effective for clinical screening where minimizing false negatives is essential.

Overall, the findings highlight that the dataset possesses strong discriminative characteristics suitable for automated classification, and that deep convolutional architectures like DenseNet121 and ResNet18 offer clinically reliable, computationally efficient, and highly accurate solutions for malaria cell image detection.

1. Introduction

Problem domain

Malaria is a life-threatening disease caused by a parasitic infection transmitted by Anopheles mosquitoes (WHO, 2024). The disease is particularly prominent in the WHO African region, which accounts for 94% of malaria cases and 95% of malaria-caused deaths globally since 2020 (WHO, 2024). According to WHO (2024), there are four main human malaria parasite species; among them, Plasmodium falciparum and Plasmodium vivax account for majority of malaria cases. P. falciparum is the deadliest, causing severe complications such as acute organ failure in patients which quickly lead to death (Zekar & Sharman, 2023).

The detection of malaria can be carried out with rapid blood tests; however, the final diagnosis is usually confirmed by blood smear microscopy (Stanford Health Care, n.d.). Unfortunately, this manual diagnostic testing can be time-consuming and requires a certain level of relevant experience and expertise. These resources are limited in WHO African region countries where malaria is most prevalent; thus, the lack of prompt diagnosis and treatment has resulted in many deaths.

To address this problem, this project aims to develop an automated system for detecting the presence of malaria parasites in microscopic blood smear images. Using deep learning, the aim is to create an efficient tool to support healthcare professionals in the prompt diagnosis of malaria. By providing accurate and widely accessible diagnostic support, this project hopes to aid in reducing delays in malaria treatment. Thus, improving patient outcomes in regions where skilled microscopy is limited.

Data overview

The dataset used for this project is originally sourced from the website of National Institutes of Health (NIH) (n.d.), accessed via a compiled version on Kaggle (Arunava, 2018). This is an image dataset of microscopic red blood cell images obtained from Giemsa-stained thin blood smear slides from the Malaria Screener research activity. There are two main classes of cell images, namely the uninfected cells and the parasitized cells. These images were collected from uninfected patients and patients infected with the P. falciparum malaria parasites.

This dataset is a single image folder named *cell_images*, in which contains two subfolders: *Parasitized* and *Uninfected*, consisting of the parasitized cell images and uninfected cell images respectively. There are 13779 images in each subfolder, totalling to 27558 images collectively.

Expectation and approach

We expected to achieve high classification accuracy using a CNN on this dataset, as the microscopic images of the two classes (*Parasitized* and *Uninfected*) are visually relatively distinct. The model is anticipated to learn how to discriminate between cells by identifying the presence of parasitic patterns in the cell images.

Several challenges were present in this project. Firstly, the raw cell images were of varying sizes and contained misclassifications. To overcome this, an alternative dataset that had undergone professional relabelling was used to filter the original dataset. Next, data preprocessing was conducted to

standardize all the images, preparing it for CNN inputs. Secondly, to increase the generalizability of the models without sacrificing real-world applicability, we applied data augmentation to the training set only. Thirdly, as this is a medical classification problem, we must ensure both the parasitized and uninfected cell images are classified correctly to avoid misdiagnoses. To address this, we first carried out comprehensive EDA on the pre-processed dataset to determine the most suitable approach for modelling. Next, we trained, fine-tuned hyperparameters, and modified the training backbone of multiple CNN models. This included leveraging transfer learning models like ResNet-18, ResNet-50, DenseNet-121, and ConvNeXt Base. Then, we evaluated these models' performance on unseen data (testing set) using various metrics such as accuracy, recall, precision, F1-score, and ROC-AUC.

2. Data Quality

Data preview

This dataset contains microscopic images of parasitized cells and uninfected cells (Figure 2.1). There is a total of 27558 images, evenly split between 2 classes (*Parasitized* and *Uninfected*), with 13779 belonging to each class. From figure 2.1, it is observed that the primary distinction between the parasitized cells from the uninfected cells is the presence of dark spot-like or ring-like patterns within the cell.

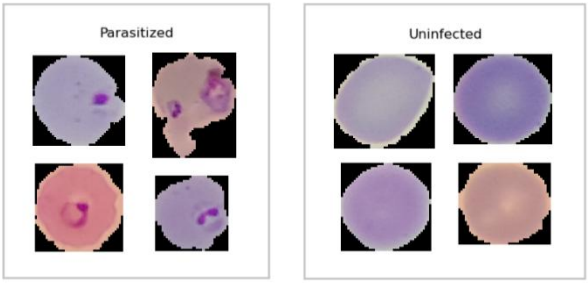

Figure 2.1: *Sample images of parasitized cells (left) and uninfected cells (right).*

Image properties

The cell images are RGB images, each with 3 colour channels. The pixel intensity of each image ranges between 0 to 255. The image dimensions vary considerably: parasitized cell images range from 46x79 to 394x241 pixels, while uninfected cell images range from 49x58 to 247x139 pixels. These varying image sizes may pose an issue in subsequent CNN modelling which typically requires a fixed input size.

Quality issue: Mislabelled images

From inspection of the raw image data, there are several images which appeared to be in the wrong class, i.e. there is presence of parasitic patterns but the image is classified as *Uninfected*, and vice versa (Figure 2.2). Similar observations have been noted by other users of this dataset on the Kaggle dataset source page.

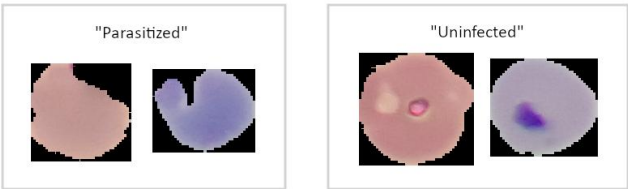

Figure 2.2: *Potentially misclassified cell images in each class.*

To address this issue, the most reliable approach is professional verification of the mislabelled images. This method is preferred over computer-based identification and correction, as inaccurate reclassification could propagate further errors within the dataset and ultimately degrade real-world model performance. This is especially important in the medical setting.

Upon further investigation, a prior study by Fuhad et al. (2020) had also identified this issue. Hence, Fuhad et al. had a professional re-label the dataset to identify mislabelled images, and they provided the updated dataset (with mislabelled images removed) for public use. However, the updated dataset

link was no longer accessible at the time of this project. Instead, an alternative version uploaded to Kaggle by another user (mlmaniacofece, 2022), provided a pre-processed version of the updated dataset in which the original file names were preserved.

Thus, the image file names in this alternative dataset were extracted and used to filter our original dataset. This ensured that only correctly labelled images, in its original form (without any preprocessing), were retained. As a result, approximately 10% of images were removed from the original dataset. The remaining 13131 *Parasitized* images and 13028 *Uninfected* images was used for subsequent analysis in this project.

Data preprocessing and augmentation

Proper data preprocessing is crucial to ensure the data is suitable for reliable CNN model training. First, all the images were pre-processed by resizing to a uniform size of 256x256 pixels. Then, each image was center-cropped to 224x224 pixels. This addressed the varying original image dimensions, providing consistent inputs for CNN while retaining only the central, most informative regions of the image. Lastly, the image pixel intensity values were normalized to range between -1 and 1. With the normalized values roughly centred around 0, this enables faster and more stable neural network training by avoiding bias towards one side of activation functions like ReLU.

Next, data augmentation helps enhance the dataset by increasing the variability of data seen by the model during training, thus improving model generalization. The data augmentation transformations are randomly applied to the images. For this project, the transformations included: horizontal flips, rescaling (zoom-in up to 20% larger), 10° clockwise and anticlockwise rotations, increasing the contrast, and sharpening the images by a factor of 2.0.

However, to simulate realistic scenarios, these transformations were restricted to the training set, leaving the validation and testing sets only minimally pre-processed. The details of training-validation-testing split can be found in the Model Development section.

3. Model Development

Overall Approach in Exploratory Data Analysis (EDA)

Prior to the model development, a series of EDA was conducted to further observe the qualitative data, denoting the nature of the data that were in images. This included class distribution analysis, pixel intensity distribution analysis, principal component analysis (PCA) and Kullback-Leibler (KL) divergence.

The analyses were implemented aiming to observe the overall distribution of the data in terms of class balance, pixel intensity variation and dimensionality structure. Specifically, they were aimed to determine the most suitable approaches for modelling, to avoid computational overhead, as well as aiming the optimal strategy for data interpretation and feature representation. Moreover, this ensured that the model development was grounded in comprehensive understanding of nature properties of the dataset.

Class Distribution Analysis

To assess the class balance of the dataset, a chi-squared test for goodness of fit was conducted, with hypotheses with null hypothesis (H_0) stating that the two classes were equally distributed and the alternative hypothesis (H_A) stating that significant differences existed between class frequencies. The approach could provide a formal observation to evaluate class balance prior to the model training to ensure that the analyses were not favouring to any class (Rahman et al., 2021).

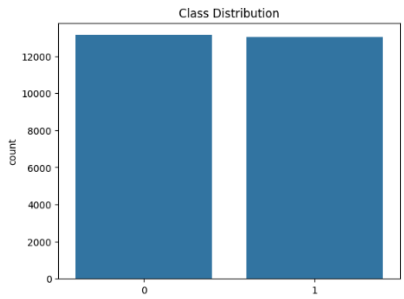

Figure 3.1: *Class Distribution Analysis*

The chi-squared test gained a p-value of 1.0, while Figure 3.1 plotted the distribution of each class, which indicated that there exist nearly equal number of samples in both classes, which further indicated that there exist no statistically significant differences in class frequencies. The balanced class distribution could suggest that the model tend to not bias toward either class during training, which thereby reducing the necessity of resampling or adjustment of weight during the training (Rahman et al., 2021).

Pixel Intensity Distribution Analysis & KL-Divergence Analysis

To investigate whether the mean pixel intensity of images in between of 2 classes could differ significantly, a two-sample t-text was conducted, in which the null hypothesis (H_0) stating that the mean pixel intensities of uninfected and parasitized cells were equal, whereby the alternative hypothesis (H_A) stating that significant difference exist between them. The t-test isa parametric statistical method to compare the means of 2 independent group, which assumption were made that their data follow approximately normal distributions (Anorboev et al., 2025; Rahman et al., 2021).

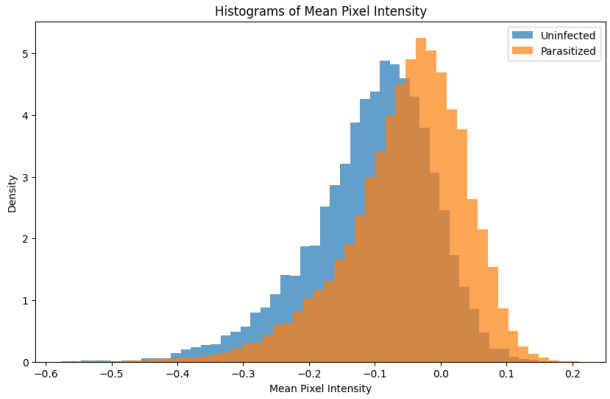

Figure 3.2: *Histogram of Mean Pixel Intensity*

Upon analysis, a p-value of < 0.00 and the KL-divergence of 0.0594 were gained. Figure 3.2 showing the histogram of mean pixel intensity among 2 classes further suggested that the assumption was accurate, denoting the approximately-normal-distribution shape showing up in the figure. Both classes showed up similar yet slightly shifted pixel intensity distribution, while the p-value of 0.00 further suggested that cell images was statistically significant compared to significance level of 0.05. This suggested that two groups possess distinguishable intensity characteristics that allow models to extract meaningful features for model training and classification (Anorboev et al., 2025).

The KL divergence that calculated the quantify the degree of dissimilarity between the two-pixel intensity distributions, measured how much information is lost when one probability distribution is used to approximate another. The KL-divergence value of 0.0594 indicated a small divergence that suggested that although both distributions shared a similar overall shape, but there exists only subtle difference in pixel intensity patterns, that further suggesting the ability of the dataset to provide discriminatory information to distinguish between each of the classes in modelling stages (Anorboev et al., 2025).

Principal Component Analysis (PCA)

To further explore the underlying structure of the dataset, PCA, which is a linear dimensionality reduction technique that could transform high-dimensional data into lower-dimensional space by identifying new orthogonal variables. The image data were flattened into feature vectors before applying PCA, and the first 2 principal components were plotted to observe potential separability between the uninfected and parasitized classes (Rahman et al., 2021).

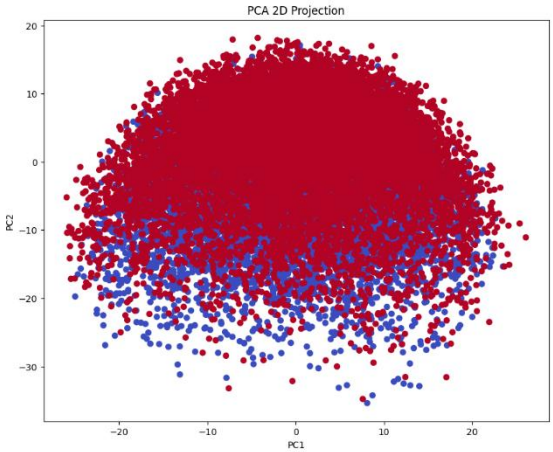

Figure 3.3: *PCA Projection of Data (Red indicating Class 1 and Blue indicating Class 0; Var=0.95)*

Figure 3.3 showed the result of PCA, which aimed to observe the projection of the data into 2-dimensional projection. From the figure it could be seen a high degree of overlap between 2 classes, that further supported the state that there exists subtle difference in terms of pixel-level features. In other words, the variations were not linearly separable in low-dimensional space. This implied that simple linear models may struggle to distinguish between the two classes. Thus, non-linear or deep learning approaches would be required to capture more complex feature representation in our dataset.

Modelling Approaches

A series of modelling approaches were conducted to evaluate the performance of different deep learning architectures for malaria cell image classification. It was aimed to build a reliable baseline and enhance its performance by conducting optimization of the architecture. The dataset was separated into 70/20/10 training-validation-testing set to ensure robust model training and evaluation and prevention of overfitting. To ensure comprehensive evaluation of the models, all models were assessed using metrics. The accuracy and AUC (Area Under the ROC Curve) were used to measure overall predictive performance while precision, recall and F1-score were used to evaluate the ability of the model to deal with false positives and false negatives.

Baseline Model

The baseline model was set as Convolutional Neural Network (CNN) due to its ability to capture spatial features and dependencies in image data. CNN could learn relevant visual features such as edges, textures and shapes through convolutional layers, which made it the best candidate to be set as baseline model. It was trained for 25 epochs using the Adam optimizer with cross-entropy loss.

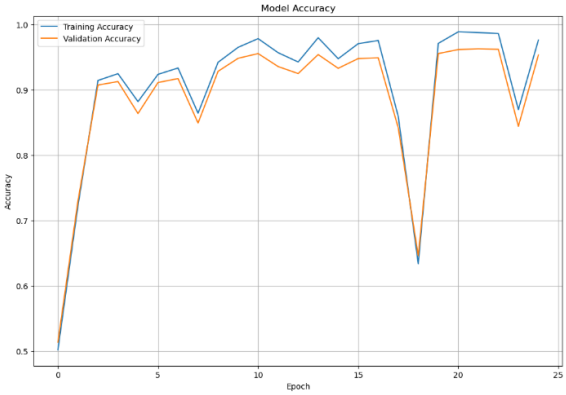

Figure 3.4: *Baseline Model Accuracy*

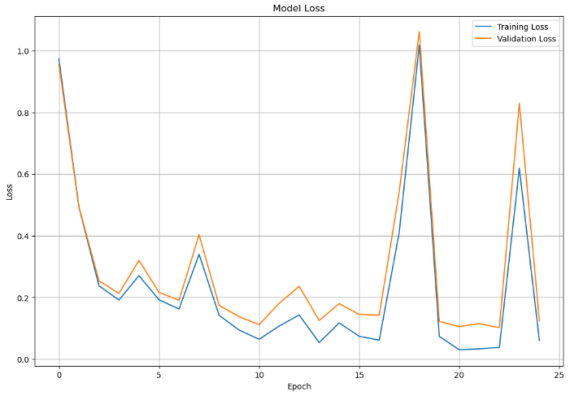

Figure 3.5: *Baseline Model Loss*

Figure 3.4 and Figure 3.5 showed the visualization of accuracy and loss of the baseline model respectively. It could be observed that the validation accuracy gained was approximately 0.95 indicating excellent performance.

Table 3.1: *Tabulation of Baseline Model Performance*

Set	Epochs	Loss	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Training	25	0.0610	0.9977	0.9966	0.9987	0.9976	0.9997
Validation		0.0862	0.9885	0.9864	0.9909	0.9886	0.9970

Table 3.1 tabulated the performance of the baseline model across training and validation sets. The model demonstrated excellent overall performance, with consistently high accuracy, precision, recall, F1-score and ROC-AUC values. The result suggests that CNN could effectively capture key patterns from the images, which indicated its suitability as a baseline model for the task.

However, there exist some minimal gap between training and validation accuracy which suggested tendency towards overfitting. Thus, consequently, there could be additional approaches to be done to further improve generalisability and robustness of the model to ensure the progressive model optimization.

Advanced Training Backbone

Model generalization and robustness are highly dependent on a well-designed training backbone. Elharrouss et al. (2024) asserted that modifying the backbone helps model to specialize in domain-specific features rather than relying heavily on generic low-level representation. The main intuition of taking this approach to model the malaria images is because the morphological characteristics of infected and healthy cells differ mainly on the existence of a red patch in the cells, which is not a feature strongly emphasized in pre-trained neural networks. Hence, a couple of advanced features such as early stopping, gradient clipping, label smoothing, cosine annealing learning rate scheduling, and mix-up data augmentation are integrated into the initial training loop. Detailed codes are provided in Appendix A.

Early Stopping and Best Weight Preservation

Early stopping is a crucial feature to monitor validation loss by halting training when performance ceases to improve. The rule of thumb is to stop the training process once validation loss fails to decrease for a user specified number of epochs. We set this number as a parameter 'patience' and the patience used is 7 for 25 epochs. This mechanism is particularly significant because it prevented the model from memorizing or overfitting the data when the validation accuracy started to plateau after reaching its optimal capacity. By implementing early stopping, it is proved that the model will be more robust to noise and prevent overparameterized neural networks (Li et al., 2020).

Moving forward, a robust weight preservation framework is also integrated with the early stopping feature. This step is essential to preserve the best-performing model weights when the validation loss reaches its minimum. Only optimal weights will be reloaded during the evaluation stage, which not only saves time but also ensures that the model loaded will be the model with the best achieved generalization performance. This mechanism is particularly critical because only the model with the most robust stage of training will be used, preventing unnecessary overtraining and overfitting.

Gradient clipping

Gradient exploding is a common issue that occurs in neural networks when the calculated gradient is extensively large during backpropagation. This can be proved by drastic changes and spikes in loss as shown in Figure 3.5. To combat this issue, gradient clipping is applied for each training iteration to ensure gradients are always clipped or remained in a stable magnitude range. As emphasized by Zhang et al. (2019), gradient clipping can accelerate training and ensure gradient smoothness across the whole training process, with avoided fluctuations in loss. In our case, we restricted the L2 norm of gradients to a maximum threshold of 1.0 thus avoided large oscillations in parameter updates and successfully maintained a consistent loss convergence shown in the Results section.

Label smoothing

Another vital feature to prevent our model from becoming overconfident in its predictions is implementing label smoothing. Instead of training on strict one-hot-labels, each target vector will be softened into a weighted average of the hard targets and the uniform distribution over labels (Müller et al., 2019). To this extent, we used a label smoothing with a factor 0.1, which means the model assigns only a probability of 0.9 to the true class and distributes the remaining 0.1 probability uniformly across all other incorrect classes. This approach is extremely useful to help our model avoid overconfident misclassification in borderline cases where cell morphology between infected and healthy cells may overlap or look similar.

Cosine Annealing Learning Rate Scheduler

Model learning rate is also a parameter that requires meticulous modulation. Hence, a learning rate scheduler is implemented to decay the learning rate throughout the whole training process. You et al. (2019) stated that a model should start with a large learning rate during early training for broad exploration then slowly decays to smaller steps in the latter epoch for fine tuning and loss convergence

to the global minimum while avoiding oscillations. The issue is how the learning rate should be scheduled and decayed in an optimal way. In this scenario, we employed the cosine annealing for a periodic shift of learning rate following a cosine curve. This not only prevents abrupt learning rate decay but also helps the model escape shallow local minima since the learning rate is oscillating in a cosine manner (Johnson et al., 2023). Hence, this approach improves our model generalization where the global minimum of the model loss will always be achieved.

Mix up Data Augmentation

Finally, data mix up was implemented at the data loading stage to reduce memorization and improve performance on unseen data. Data Mixup is proposed by Zhang et al. (2017) where pair of training samples and their labels are combined using a mixing coefficient $\lambda \sim \text{Beta}(\alpha, \alpha)$ with $\alpha = 0.2$ and 0.4 in our case. Each new input will then be generated as $\tilde{x} = \lambda x_i + (1 - \lambda)x_j$ and $\tilde{y} = \lambda y_i + (1 - \lambda)y_j$ where x is the raw input vectors while y is the one-hot label encodings. This approach is used because the malaria dataset does not have diverse samples as the images are relatively simple where overfitting is likely to happen. Mixup encourages the model to learn smoother decision boundaries by enforcing approximate linearity between training samples, thus boosting the generalization ability of the model. In other words, this is a technique that generates new labels by mixing or blending two existing samples to enhance our model's ability to unseen data.

Transfer learning

After modifying the training backbone, we proceeded to build more advanced models. Transfer learning is then being adapted. According to Pan and Yang (2010), transfer learning is a strategy to leverage pre-trained model that is trained prior on a large dataset and apply it to a domain specific task. The knowledge gained by the pre-trained model is in the form of learned features in the early layers of network will be transferred to our task, which is the malaria infected cells image classification. We chose to utilize this technique because transfer learning can reduce training time and is much more efficient in fine-tuning the model to adapt to our image classification problem. Furthermore, the pretrained weights can act as a form of regularization where the model is pretrained on large ImageNet dataset previously. This prevents overfitting to any spurious noises in the malaria dataset. This approach was chosen because Salehi et al. (2023) has proven that many medical imaging studies achieved better generalization when using transfer learning in real world practices. Detailed codes are in Appendix B.

After pretrained model initialization, we implemented an extremely crucial procedure in transfer learning, which is the freezing and unfreezing layers of pretrained model then fine tune it. Freezing layers or warming up layers means that we set all parameters to be non-trainable and prevent gradients computed or weights updated via back propagation so the pretrained weights can be retained. The reason behind freezing layers is because early layers of a neural network are often learn basic or generic features and fine-tuning them too early may lead to overfitting to noises (Guo et al., 2019). Therefore, we freeze the layers for 5 epochs, where pretrained weights are intact and only classifier heads are adjusted. The significance of this approach is to give the newly added layers time to adapt and stabilize, hence it will be easier to be trainable again in the unfreezing phase. For 25 epochs we implemented the unfreezing stage where all layers are trainable and further backpropagation is enabled to adjust

weights to suit our malaria infected cell classification task. In other words, this is a fine tune stage where we set a smaller learning rate of 0.0002 to let the entire neural network refine lower-level features to the malaria images. We chose this freezing-unfreezing approach in modelling because Ayana et al. (2024) have shown that this staged strategy is effective in retaining general and robust features while adapting to the domain-specific features.

ResNet18 & ResNet50

Residual Networks, also known as ResNet, is the first pretrained model that we used. Although proposed quite some time by He et al. (2015), it is still a classic yet effective CNN model that introduced residual or skip blocks to allow deeper networks to be trained without the gradient vanishing problem. This model was chosen for transfer learning because the skip connections help to stabilize training, especially during the fine-tuning stage to capture deeper features in the malaria images. In this context, both shallower ResNet18 with 18 layers and deeper ResNet50 with 50 layers are pretrained to capture mid-level image features which are useful to distinguish subtle parasitic (malaria) features.

DenseNet121

Densely Connected Convolutional Networks (DenseNet) is proposed by Huang et al. (2016) where all layers are connected with the concatenation of the feature maps of all preceding layers and are passed in a feed forward manner. This feed-forward CNN was chosen as our second pretrained model for transfer learning because it enables stronger feature propagation since each layer has direct access to the preceding layers, and these features can be reused for parameter efficiency. An effective set of reusable features can help the network retain low-level discriminative signals across layers, where even the slightest feature in the malaria dataset can be captured.

ConvNeXt-base

The final pretrained model used was the ConvNeXt-base, which is introduced as a modernized convolutional architecture that is inspired by principles from Vision Transformers (ViTs), proposed by Liu et al. (2022). In this model, larger kernel sizes, inverted bottlenecks and architectural adjustments are implemented to match the transformer-based networks' scaling behaviour. This hybrid architecture is much more sophisticated than the traditional CNN, thus it was chosen by us to capture subtle morphological patterns in high-resolution cell images. In addition, this model can leverage more expressive features to enable finer discrimination between infected and healthy cells, which boosts the confidence of the model in the image classification task even when images from both classes look similar.

Key Assumptions and Potential Limitations of Models

While the integration of advanced training backbone and transfer learning is a strong combination and effective strategy in our malaria cell image classification task, there are several assumptions and potential limitations are acknowledged by us to provide a realistic perspective of the expected models performance.

First, all implemented models assume that the malaria cell images are accurately labeled. This is addressed in the data quality part, where the mislabeled images are all correctly labeled after some research. Furthermore, all the advanced methods we utilize require large amounts of data to reach its optimal performance. This is not an issue since we have 13779 images each for both infected and healthy cells thus the assumption of large data requirement is fulfilled. Moving on, there is a potential limitation of domain shift during transfer learning. The reason behind this is because the pretrained features from ImageNet is assumed to be generalized well to the malaria microscopy domain in our case. In reality, the domain shift of differences in colour, texture and structure between different image domains may limit the transferability of learned filters. This limitation is combatted with our implementation of three different pretrained models which provides diversity in low, medium and high-level feature representations to enhance model generalization and robustness. Finally, since we trained on several large models, these advanced architectures will trade off performance with expensive computational resources. Hence, we assume that a strong GPU with sufficient memory is used in our image classification task.

4. Results

Training Dynamics

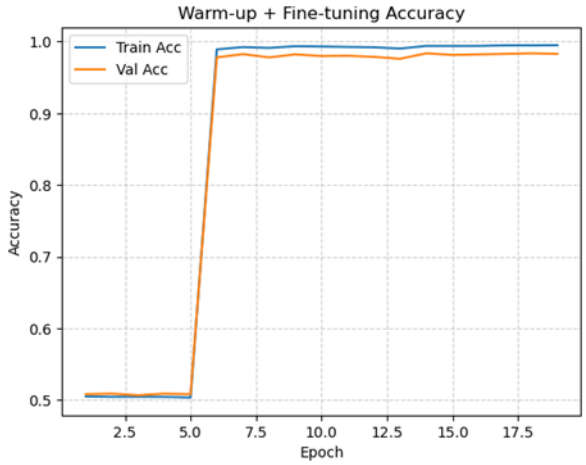

Figure 4.1: *ResNet18 Training and Validation Accuracy over 19 epochs*

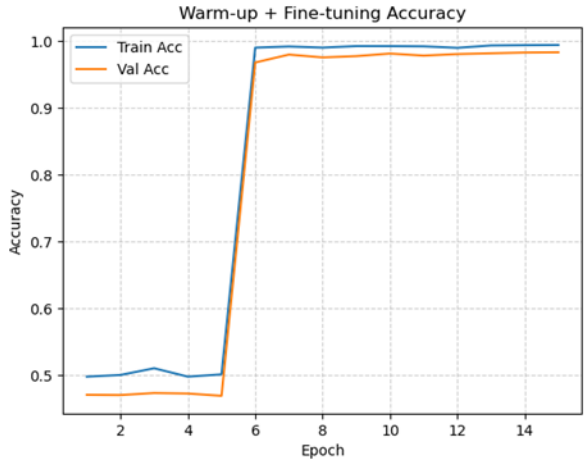

Figure 4.2: *ResNet50 Training and Validation Accuracy over 15 epochs*

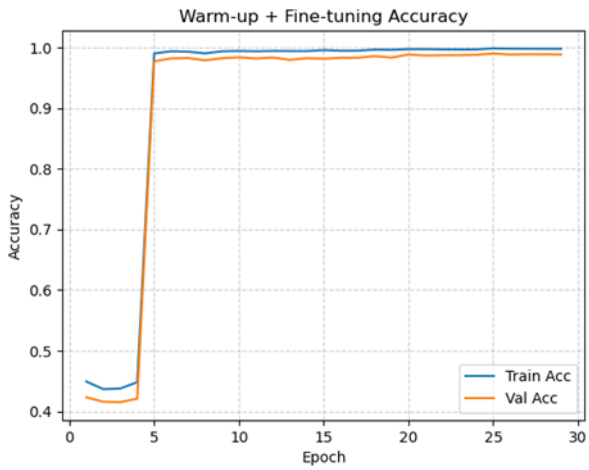

Figure 4.3: *DenseNet121 Training and Validation Accuracy over 29 epochs*

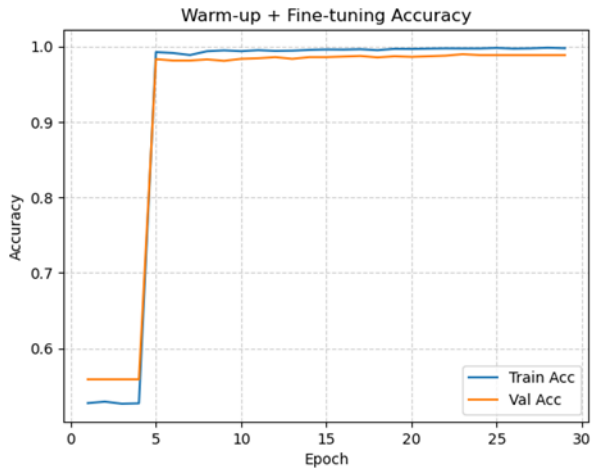

Figure 4.4: *ConvNeXt Base Training and Validation Accuracy over 29 epochs*

The ResNet18, ResNet50, DenseNet121 and ConvNeXt Base CNN as shown in Figure 4.1 to Figure 4.3 demonstrated a two-stage learning pattern, validating the effectiveness of our transfer learning approach.

Warm-up Phase (Epochs 1-5)

The models showed varying baseline validation accuracy while the backbones were frozen. ResNet18 and ResNet50 started at approximately 50% and 45% respectively. This shows us that the models both had near random performance for binary classification. DenseNet121 had the lowest

baseline validation accuracy of about 43%, likely due to its dense connectivity requiring more classifier adaptation. ConvNeXt Base achieved the highest warm-up validation accuracy at approximately 56%, suggesting its modern design elements transfer more effectively from ImageNet.

Fine-tuning Phase (Epoch 6 onwards)

Upon unfreezing all layers, every model exhibited explosive learning with dramatic improvements within a single epoch. Both the validation and training accuracy jumped to about 98-99% across all models. This indicates that the feature adaptation across all layers occurred rapidly.

Convergence and Stability

All models stabilized at approximately 99% training and 98% validation accuracy. The tight train-validation coupling indicates excellent generalization without overfitting. DenseNet121 is the most stable across the four models, with a near-perfect overlap maintained across 35 epochs. ConvNext Base also exhibited similar stability despite its significantly larger size of 88.6 million parameters. This small gap validates that our regularization strategies effectively prevented models from memorizing training data.

Model Performance (Validation)

Figure 4.5: *Validation Set Performance Heatmap*

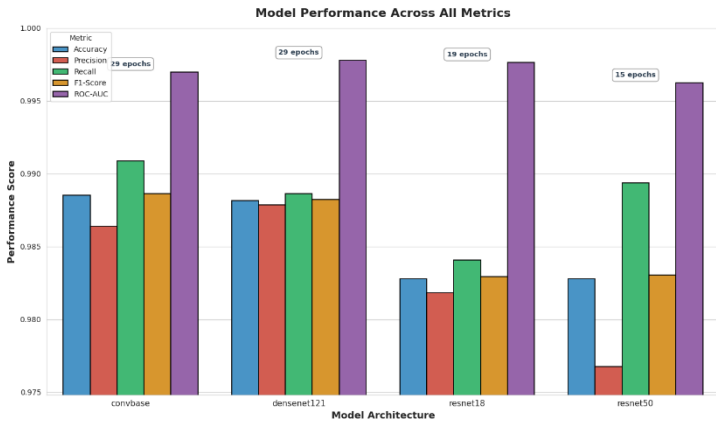

Figure 4.6: *Validation Set Performance Comparison Across Four CNN Architectures*

All four models achieved excellent validation accuracy ranging from 98.28% to 98.85% as shown in Figure 4.5. The narrow 0.57 percentage difference indicates that there is consistent learning across architectures, with performance closely matching subsequent training and test set results. ConvNeXt Base achieved the highest validation accuracy at 98.85% followed by DenseNet121 (98.81%), ResNet18 (98.28%) and ResNet50 (98.28%). The ResNet architectures showed identical validation accuracy despite ResNet50 having 2.2 times more parameters, demonstrating diminishing returns of increased model depth.

The DenseNet121 model obtained the best validation precision of 98.79%, indicating the fewest false positives during validation. ConvNeXt Base (98.64%) and ResNet18 (98.18%) followed, while

ResNet50 showed the lowest precision of 97.68%. The 1.11 percentage point range from 97.68% to 98.79% suggests meaningful differences in false positive rates across the models during validation. ConNeXt Base achieved the highest validation recall at 99.09%, correctly identifying the most infected cells during validation. ResNet50 (98.94%), DenseNet121 (98.86%) and ResNet18 (98.41%) followed closely. All models exceeded the 98% threshold, this indicates strong recall across all four models with recall ranging from 98.41% to 99.09%.

ConvNeXt Base led with a validation F1-score of 98.86%, followed by DenseNet121 (98.82%), ResNet50 (98.30%), and ResNet18 (98.29%). The tight clustering (range: 98.29% to 98.86%, 0.57 percentage points) demonstrates balanced precision-recall trade-offs, with all models maintaining F1-scores above 98.2%. All models demonstrated near-perfect discrimination with validation AUC-ROC scores ranging from 99.62% to 99.78%. DenseNet121 achieved the highest AUC-ROC of 0.9978, followed by ResNet18 (0.9976), ConvNeXt Base (0.9970) and ResNet50 (0.9962). The minimal 0.16 percentage point range indicates that all four models are equally good at distinguishing between categories despite their different structures.

Each model required different validation durations to reach optimal performance based on early stopping criteria to prevent overfitting. ConvNeXt Base and DenseNet121 both converged at 29 epochs (Figure 4.6), while ResNet18 and ResNet50 required 19 and 15 epochs respectively. Despite fastest convergence, ResNet50 achieved the lowest precision score, this shows that rapid convergence does not guarantee optimal performance.

Each model required different validation durations to reach optimal performance. ConvNeXt Base and DenseNet121 both converged at 29 epochs (Figure 4.6), while ResNet18 and ResNet50 required 19 and 15 epochs respectively. Despite fastest convergence, ResNet50 achieved the lowest precision score, this shows that rapid convergence does not guarantee optimal performance. Conversely, DenseNet121's longer training duration of 29 epochs resulted in the highest precision score and has the most balanced metric profile.

The multi-metric visualization in Figure 4.6 reveals distinct model profiles. While all four models demonstrate robust capability, DenseNet121 achieves the highest scores across nearly all metrics, indicating a marginal but consistent superiority in both discriminative power and classification balance. ResNet18 also achieves high ROC-AUC but shows slightly lower accuracy and precision bars. ConvNeXt Base exhibits strong overall performance with the highest recall while ResNet50 displays the most imbalanced profile, with the lowest precision (97.68%) contrasting sharply with its competitive recall (98.94%) highlighting a notable precision-recall trade-off during validation.

Model Performance (Testing)

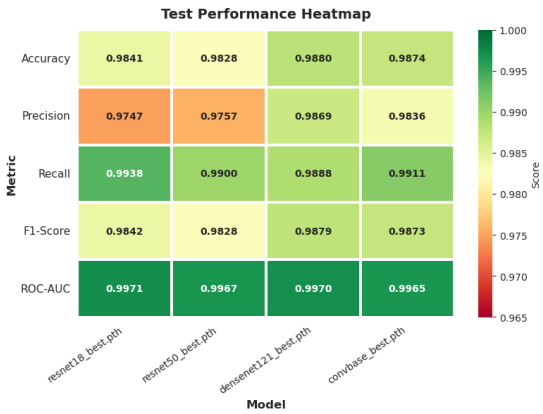

Figure 4.7: *Testing Set Performance Heatmap*

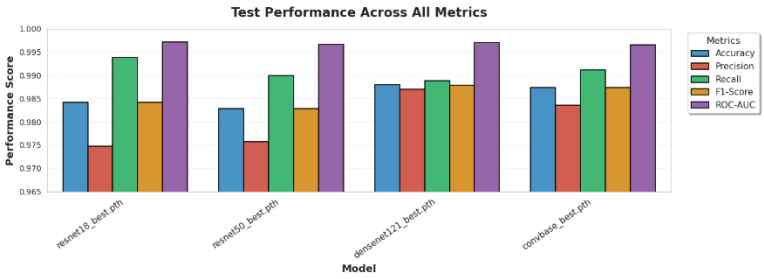

Figure 4.8: *Testing Set Performance Comparison Across Four CNN Architectures*

All four models were evaluated on a held-out test set of 5233 images as shown in Figure 4.7. All models achieved excellent test accuracy ranging from 98.28% to 98.80%. The narrow 0.52 percentage point difference indicates that all models successfully learned robust features for malaria detection. DenseNer121 achieved the highest accuracy at 98.80%, followed closely by ConvNeXt Base (98.41%), ResNet18 (98.41%) and ResNet50 (98.28%). Despite the substantial differences in model size (8 million to 88.6 million parameters), accuracy only varied by 0.52 percentage points. This shows that increasing model complexity yields diminishing returns.

DenseNet121 exhibited the best test precision of 98.69%, indicating the fewest false positives among all models. ConvNeXt Base (98.36%) followed, while ResNet50 and ResNet18 showed lower precision of 97.57% and 97.47% respectively. The 1.22 percentage point range from 97.47% to 98.69% suggests meaningful differences in false positive rates across models. ResNet18 achieved the highest test recall at 99.38%, missing only 0.62% of infections. This represents the most critical metric for clinical deployment, as false negatives (missed infections) can be fatal. ConvNeXt Base (99.11%) and ResNet50 (99%) also demonstrated excellent recall, while DenseNet121 (98.88%) showed slightly lower but still clinically acceptable recall. All models exceeded the 98% threshold with recall ranging from 98.88% to 99.38%.

The harmonic mean of precision and recall showed DenseNet121 leading at 98.79%, followed by ConvNeXt Base (98.73%), ResNet18 (98.42%) and ResNet50 (98.28%). The tight clustering of F1-scores indicates that all models achieved balanced performance between precision and recall. All models demonstrated near-perfect discrimination capability with test AUC-ROC scores ranging from 99.65% to 99.71%. ResNet18 achieved the highest AUC-ROC of 0.9971 followed by DenseNet (0.9970), ConvNeXt Base (0.9967) and ResNet50 (0.9965). The minimal 0.06 percentage point range indicates equivalent threshold-independent performance across all architectures. These exceptional

scores demonstrate that all models can reliably distinguish between infected and uninfected cells across all classification thresholds.

DenseNet121 emerged as the most efficient model, achieving the highest test accuracy (98.80%) with only 8M parameters. ConvNeXt Base required 88.6M parameters (11× more) but achieved 0.06% lower accuracy. ResNet18 offered the optimal balance, combining competitive accuracy (98.41%), highest recall (99.38%) and moderate size (11.7M parameters). Model selection depends on clinical priorities. ResNet18's highest recall (99.38%) makes it ideal for primary screening where catching all infections is critical. DenseNet121's balanced performance (98.80% accuracy, 98.69% precision) suits laboratory confirmation and quality assurance settings where both sensitivity and specificity matter.

The multi-metric visualization in Figure 4.8 reveals distinct profiles. DenseNet121 demonstrates the most balanced performance with highest accuracy and precision. ResNet18 achieves highest recall, optimal for screening applications. ConvNeXt Base shows strong performance but cannot justify its 11 times parameter overhead. ResNet50 displays imbalanced metrics with no advantages over the smaller ResNet18.

Models rank as: (1) DenseNet121 - best overall balance and efficiency, (2) ResNet18 - best recall and speed, (3) ConvNeXt Base - good performance but inefficient, (4) ResNet50 - no advantages over smaller models. All models exceed clinical requirements which it is required for the accuracy and recall score to be more than equal to 93%.

5. Conclusion

In conclusion, this project has successfully demonstrated the effectiveness of deep learning techniques to automate detection of malaria-infected cells from microscopic images. Through comprehensive data preprocessing, including grayscale normalization and dimensionality reduction via PCA, the dataset was optimized for model training and analysis. The exploratory data analysis confirmed balanced class distribution and meaningful visual distinctions between infected and uninfected cells.

The modelling phase began with a baseline CNN that achieved strong performance in both training and validation. To enhance robustness and prevent overfitting, training strategies such as early stopping, gradient clipping, label smoothing, cosine annealing learning rate scheduling, and mix-up data augmentation were employed. These techniques improved generalization and model stability, thus making the model perform well even beyond the training data.

We improved this foundation by utilizing transfer learning with multiple pretrained architectures, including ResNet18, ResNet50, DenseNet121, and ConvNeXt Base. All transfer learning models achieved excellent test performance, with accuracies consistently exceeding 98%. DenseNet121 delivered the most balanced results, achieving the highest test accuracy while maintaining computational efficiency with fewer parameters. On the other hand, ResNet18 had the highest recall score, making it particularly valuable for clinical applications where minimizing false negatives is essential.

Overall, the findings highlight that deep learning provides an efficient, accurate, and scalable solution for malaria detection. With further optimization and validation on larger, real-world datasets, such models could be integrated into clinical decision-support systems to assist healthcare professionals in early and reliable malaria diagnosis.

References

Anorboev A., Anorboeva, S., Musaev, J., Usmanov E., Hwang, D., Seo, Y.-S., & Hong, J. (2025). Selective Intensity Ensemble Classifier (SIEC): A Triple-Threshold Strategy for Microscopic Malaria Cell Image Classification. *IEEE Access*, *13*, 101609–101623. https://doi.org/10.1109/access.2025.3574528

Arunava, C. (2018, December 5). *Malaria Cell Images Dataset* [Data set]. Kaggle. https://www.kaggle.com/datasets/iarunava/cell-images-for-detecting-malaria

Ayana, G., Dese, K., Abagaro, A. M., Jeong, K. C., Yoon, S., & Choe, S. (2024). Multistage transfer learning for medical images. *Artificial Intelligence Review, 57*(232). https://doi.org/10.1007/s10462-024-10855-7

Elharrouss, O., Akbari, Y., Almaadeed, N., & Al-Maadeed, S. (2024). Backbones-review: Feature extraction newtworks for deep learning and deep reinforcement learning approaches. *Computer Vision and Pattern Recognition, 53*. https://doi.org/10.1016/j.cosrev.2024.100645

Fuhad, K. M. F., Tuba, J. F., Sarker, Md. R. A., Momen, S., Mohammed, N., & Rahman, T. (2020). Deep learning based automatic malaria parasite detection from blood smear and its smartphone based application. *Diagnostics, 10*(5), 329. https://doi.org/10.3390/diagnostics10050329

Guo, Y., Shi, H., Kumar, A., Grauman, K., Rosing, T., & Feris, R. (2019). SpotTune: Transfer learning through adaptive fine-tuning. *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR),* 4800-4899. https://doi.org/10.1109/CVPR.2019.00494

He, K., Zhang, X., Ren, S., & Sun, J. (2015). Deep residual learning for image recognition. *2016 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR),* 770-778. https://doi.org/10.1109/CVPR.2016.90

Huang, G., Liu, Z., van der Maaten, L., & Weinberger, K. Q. (2016). Densely connected convolutional networks. *2017 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR),* 2261-2269. https://doi.org/10.1109/CVPR.2017.243

Johnson, O. V., Chew, X., Khaw, K. W., & Lee, M. H. (2023). Ps-CALR- Periodic-shift cosine annealing learning rate for deep neural networks. *IEEE Access, 11*, 139171-139186. https://doi.org/10.1109/ACCESS.2023.3340719

Li, M., Soltanolkotabi, M., & Oymak, S. (2019). Gradient descent with early stopping is provably robust to label noise for overparameterized neural networks. *Proceedings of Machine Learning Research, 108.* https://doi.org/10.48550/arXiv.1903.11680

Liu, Z., Mao, H., Wu, C., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). A ConvNet for the 2020s. *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR),* 11966-11976. https://doi.org/10.1109/CVPR52688.2022.01167

mlmaniacofece (2022, February 21). *Modified malaria dataset* [Data set]. Kaggle. https://www.kaggle.com/datasets/mlmaniacofece/modified-malaria-dataset

Müller, R., Kornblith, S., & Hinton, G. (2019). When does label smoothing help? *Conference and Workshop on Neural Information Processing Systems 2019.* https://doi.org/10.48550/arXiv.1906.02629

National Institutes of Health (NIH) (n.d.). *Malaria Screener: Malaria datasets repository* [Data set]. National Library of Medicine. https://ceb.nlm.nih.gov/repositories/malaria-datasets/

Pan, S. J., & Yang, Q. (2010). A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering, 22*(10), 1345-1359. https://doi.org/10.1109/TKDE.2009.191

Rahman, A., Zunair, H., Reme, T. R., Rahman, M. S., & Mahdy, M. R. C. (2021). A comparative analysis of deep learning architectures on high variation malaria parasite classification dataset. *Tissue and Cell*, *69*, 101473. https://doi.org/10.1016/j.tice.2020.101473

Salehi, A. W., Khan, S., Gupta, G., Alabduallah, B. I., Almjally, A., Alsolai, H., Siddiqui, T., & Mellit, A. (2023). *Sustainability, 15*(7), 5930. https://doi.org/10.3390/su15075930

Stanford Health Care (n.d.). *Malaria Diagnosis*. https://stanfordhealthcare.org/medical-conditions/primary-care/malaria/diagnosis.html

World Health Organisation (WHO) (2024). Global trends in the burden of malaria. In *World malaria report 2024: Addressing the inequity in the global malaria response* (pp. 6-29). World Health Organisation. https://www.who.int/publications/i/item/9789240104440

World Health Organization (WHO) (2024, December 11). *Malaria.* https://www.who.int/news-room/fact-sheets/detail/malaria

You, K., Long, M., Wang, J., & Jordan, M. I. (2019). How does learning rate decay help modern neural networks? *International Conference on Learning Representation 2020.* https://doi.org/10.48550/arXiv.1908.01878

Zekar, L. & Sharman, T. (2023, August 8). Plasmodium falciparum malaria. In *StatPearls*. Treasure Island (FL): StatPearls Publishing. https://www.ncbi.nlm.nih.gov/books/NBK555962/

Zhang, H., Cisse, M., Dauphin, Y. N., & Lopez-Paz, D. (2017). Mixup: Beyond empirical risk minimization. *International Conference on Learning Representation 2018.* https://doi.org/10.48550/arXiv.1710.09412

Zhang, J., He, T., Sra, S., & Jadbabaie, A. (2019). Why gradient clipping accelerates training: A theoretical justification for adaptivity. *International Conference on Learning Representation 2020.* https://doi.org/10.48550/arXiv.1905.11881

Appendix A: Advanced Training Backbone

```python
def train_model(model, optimizer, train_loader, val_loader, num_epochs, patience, clip_grad=1.0,
                model_name="best_model.pth", history_name="history.csv"):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)

    # Label smoothing
    criterion = nn.CrossEntropyLoss(label_smoothing=0.1)

    # Cosine Annealing LR
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=num_epochs)

    # History dict
    history = {
        'train_loss': [], 'val_loss': [],
        'train_accuracy': [], 'val_accuracy': [],
        'train_precision': [], 'val_precision': [],
        'train_recall': [], 'val_recall': [],
        'train_roc_auc': [], 'val_roc_auc': [],
        'train_f1_score': [], 'val_f1_score': []
    }

    best_val_loss = float("inf")
    best_weights = None
    patience_counter = 0
    early_stop_epoch = None

    # Mixup
    def mixup_data(x, y, alpha=1.0):
        if alpha > 0:
            lam = np.random.beta(alpha, alpha)
        else:
            lam = 1
        batch_size = x.size(0)
        index = torch.randperm(batch_size).to(x.device)
        mixed_x = lam * x + (1 - lam) * x[index, :]
        y_a, y_b = y, y[index]
        return mixed_x, y_a, y_b, lam

    def mixup_criterion(criterion, pred, y_a, y_b, lam):
        return lam * criterion(pred, y_a) + (1 - lam) * criterion(pred, y_b)
```

Figure A1: *Code for label smoothing, cosine annealing learning rate and data mix up*

```python
            loss.backward()
            if clip_grad is not None:
                nn.utils.clip_grad_norm_(model.parameters(), clip_grad)
            optimizer.step()
```

Figure A2: *Code for gradient clipping*

```python
        # ---- Early Stopping ----
        if val_metrics['loss'] < best_val_loss:
            best_val_loss = val_metrics['loss']
            best_weights = model.state_dict()
            patience_counter = 0
        else:
            patience_counter += 1
            print(f"  No improvement. EarlyStopping counter: {patience_counter}/{patience}")
            if patience_counter >= patience:
                early_stop_epoch = epoch + 1
                print(f"Early stopping triggered at epoch {early_stop_epoch}!")
                break

    # === Save best model ===
    if best_weights is not None:
        model.load_state_dict(best_weights)

        os.makedirs("./model", exist_ok=True)
        model_path = os.path.join("./model", model_name)
        torch.save(model.state_dict(), model_path)
        print(f"Best model saved to {model_path}")
```

Figure A3: *Code for early stopping and model weight preservation*

Appendix B: Transfer learning

```python
def make_resnet18(num_classes=2, pretrained=True):
    weights = ResNet18_Weights.IMAGENET1K_V1 if pretrained else None
    model = resnet18(weights=weights)
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model

def make_resnet50(num_classes=2, pretrained=True):
    weights = ResNet50_Weights.IMAGENET1K_V1 if pretrained else None
    model = resnet50(weights=weights)
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model
```

Figure B1: *Code for transfer learning of ResNet18 and ResNet50*

```python
def make_densenet121(num_classes=2, pretrained=True):
    weights = DenseNet121_Weights.IMAGENET1K_V1 if pretrained else None
    model = densenet121(weights=weights)
    in_feats = model.classifier.in_features
    model.classifier = nn.Linear(in_feats, num_classes)
    return model
```

Figure B2: *Code for transfer learning of DenseNet121*

```python
def make_convnext_base(num_classes=2, pretrained=True):
    weights = ConvNeXt_Base_Weights.IMAGENET1K_V1 if pretrained else None
    model = convnext_base(weights=weights)
    in_feats = model.classifier[2].in_features
    model.classifier[2] = nn.Linear(in_feats, num_classes)
    return model
```

Figure B3: *Code for transfer learning of ConvNeXt-base*

```python
# === Warm-up ===
warmup_history, model, _ = train_model(
    model, optimizer, train_loader, val_loader,
    num_epochs=5, patience=3, clip_grad=1.0,
    model_name="resnet18_warmup.pth",
    history_name="resnet18_warmup_history.csv"
)

# === Unfreeze for fine-tuning ===
for p in model.parameters():
    p.requires_grad = True

optimizer = torch.optim.Adam(model.parameters(), lr=2e-4)

finetune_history, model, _ = train_model(
    model, optimizer, train_loader, val_loader,
    num_epochs=25, patience=7, clip_grad=1.0,
    model_name="resnet18_best.pth",
    history_name="resnet18_history.csv"
)
```

Figure B4: *Code for freezing and unfreezing training backbone*

Appendix C: Group Assessment Form

Group Project Assessment for ADS2002						
Studio: MA Monday 10:00						
Group: 2						
		Chow Chun Kei (33520771)	***Esher Loh Ci Ying (34392793)***	***Gue Zhen Xue (33521352)***	***Lim Hui Shawn (34017259)***	***Wesley Kong Zhe Hung (33521743)***
Coordination	The sum of this row should be 100. The entry should reflect the group member's percentage contribution to this project task.	17	21	20	19	23
Research	The sum of this row should be 100. The entry should reflect the group member's percentage contribution to this project task.	19	20	17	23	21
Coding	The sum of this row should be 100. The entry should reflect the group member's percentage contribution to this project task.	23	17	21	20	19
Report Writing	The sum of this row should be 100. The entry should reflect the group member's percentage contribution to this project task.	21	23	19	17	20
Presentation Preparation	The sum of this row should be 100. The entry should reflect the group member's percentage contribution to this project task.	20	19	23	21	17
Attendance at Final Presentation	This entry should be 1 (present) or 0 (absent)	1	1	1	1	1